

MongoDB Project Report

ONLINE MEDICINE DELIVERY

A NoSQL Database Management System Project

Submitted By

Shrishti Sonker

Roll Number: 46 / 1250258440

Submitted To

Mr. Harendra Singh

Technology: MongoDB and MongoDB Shell (mongosh)

Project Type: Database / CRUD Operations

Academic Project Report

1. Introduction

The Online Medicine Delivery project is a MongoDB-based database application designed to store and manage customer, medicine, order, payment and delivery information. The project demonstrates document insertion, retrieval, updating, deletion, sorting and aggregation.

2. Objectives

To understand NoSQL databases; create and manage an online medicine delivery dataset; perform CRUD operations; use insertOne() and insertMany(); retrieve and sort records; and perform aggregation for sales, order and payment analysis.

3. Software Requirements

MongoDB Community Server
MongoDB Shell (mongosh)
MongoDB Compass (optional)
JSON dataset
Windows / Linux / macOS

4. Database Design

Database: **medicineDeliveryDB**

Collection: **orders**

Fields may include orderId, customerName, medicineName, category, quantity, price, totalAmount, paymentMethod, paymentStatus, orderStatus, deliveryAddress and orderDate.

5. Sample Document Structure

```
{
  orderId: 1001,
  customerName: "Aarav Sharma",
  medicineName: "Paracetamol 500mg",
  category: "Tablet",
  quantity: 2,
  price: 30,
  totalAmount: 60,
  paymentMethod: "UPI",
  paymentStatus: "Paid",
  orderStatus: "Delivered",
  deliveryAddress: "Lucknow, Uttar Pradesh"
}
```

A document stores the details of one medicine order.

6. Create Database and Collection

```
use medicineDeliveryDB
db.createCollection("orders")
show collections
```

The use command selects the database and createCollection() creates the orders collection.

7. Insert One Operation

```
db.orders.insertOne({
  orderId: 1001,
  customerName: "Aarav Sharma",
  medicineName: "Paracetamol 500mg",
  category: "Tablet",
  quantity: 2,
  price: 30,
```

```

    totalAmount: 60,
    paymentMethod: "UPI",
    paymentStatus: "Paid",
    orderStatus: "Delivered",
    deliveryAddress: "Lucknow, Uttar Pradesh"
  })
})

```

Expected output contains acknowledged: true and the inserted document ObjectId.

8. Insert Many Operation

```

db.orders.insertMany([
  {orderId:1002, customerName:"Ananya Singh",
    medicineName:"Vitamin C", category:"Tablet",
    quantity:1, price:180, totalAmount:180,
    paymentMethod:"COD", paymentStatus:"Pending",
    orderStatus:"Processing"},
  {orderId:1003, customerName:"Rohan Verma",
    medicineName:"Cough Syrup", category:"Syrup",
    quantity:2, price:120, totalAmount:240,
    paymentMethod:"UPI", paymentStatus:"Paid",
    orderStatus:"Shipped"},
  {orderId:1004, customerName:"Priya Gupta",
    medicineName:"ORS Sachet", category:"Supplement",
    quantity:5, price:25, totalAmount:125,
    paymentMethod:"Card", paymentStatus:"Paid",
    orderStatus:"Delivered"}
])

```

Expected output contains acknowledged: true and insertedIds for the inserted documents.

9. Read / Find Operations

```

db.orders.find()
db.orders.find({ orderStatus: "Delivered" })
db.orders.find({ totalAmount: { $gt: 150 } })
db.orders.findOne({ orderId: 1002 })

```

These queries display all records, delivered orders, orders above Rs.150, and one selected order.

10. Update Operations

```

db.orders.updateOne(
  { orderId: 1002 },
  { $set: { paymentStatus: "Paid",
    orderStatus: "Shipped" } }
)

db.orders.updateMany(
  { orderStatus: "Processing" },
  { $set: { orderStatus: "Confirmed" } }
)

```

MongoDB returns matchedCount and modifiedCount for update operations.

11. Delete Operations

```

db.orders.deleteOne({ orderId: 1004 })
db.orders.deleteMany({ orderStatus: "Cancelled" })

```

deleteOne() removes one matching document; deleteMany() removes all matching documents.

12. Sorting Operations

```

db.orders.find().sort({ totalAmount: 1 })
db.orders.find().sort({ totalAmount: -1 })
db.orders.find().sort({ customerName: 1 })

```

1 means ascending order and -1 means descending order.

13. Aggregation - Orders by Status

```
db.orders.aggregate([
  {$group: {
    _id: "$orderStatus",
    totalOrders: {$sum: 1}
  }},
  {$sort: {totalOrders: -1}}
])
```

Groups orders by order status and counts records in each group.

14. Aggregation - Total Sales

```
db.orders.aggregate([
  {$group: {
    _id: null,
    totalSales: {$sum: "$totalAmount"}
  }}
])
```

Calculates the combined sales amount.

15. Aggregation - Average Order Value

```
db.orders.aggregate([
  {$group: {
    _id: null,
    averageOrderValue: {$avg: "$totalAmount"}
  }}
])
```

Calculates the average order amount.

16. Aggregation - Sales by Medicine

```
db.orders.aggregate([
  {$group: {
    _id: "$medicineName",
    totalQuantitySold: {$sum: "$quantity"},
    revenue: {$sum: "$totalAmount"}
  }},
  {$sort: {revenue: -1}}
])
```

Groups records by medicine and sorts medicines according to revenue.

17. Match + Group Aggregation

```
db.orders.aggregate([
  {$match: {paymentStatus: "Paid"}},
  {$group: {
    _id: "$paymentMethod",
    numberOfOrders: {$sum: 1},
    totalAmount: {$sum: "$totalAmount"}
  }},
  {$sort: {totalAmount: -1}}
])
```

Filters paid orders and summarizes them by payment method.

18. Projection

```
db.orders.find(
  {},
  {_id:0, orderId:1, customerName:1,
   medicineName:1, totalAmount:1, orderStatus:1}
)
```

Displays selected fields only.

19. Count Operation

```
db.orders.countDocuments({  
  orderStatus: "Delivered"  
})
```

Returns the number of delivered orders.

20. CRUD Summary

Operation	MongoDB Method	Purpose
Create	insertOne(), insertMany()	Insert new records
Read	find(), findOne()	Retrieve records
Update	updateOne(), updateMany()	Modify records
Delete	deleteOne(), deleteMany()	Remove records
Sort	sort()	Arrange results
Aggregate	aggregate()	Analyze and summarize data

21. Conclusion

The Online Medicine Delivery project demonstrates how MongoDB can manage flexible order data efficiently. CRUD operations manage records, while sorting and aggregation provide useful analysis of medicines, sales, payments and delivery status.